Assignment 1

Supplementary Subject in Cultural Data Science, Fall 2025

Victoria Egeskov Nielsen

5. semester, Cognitive Science Bsc

Arts, University of Aarhus

Jens Chr. Skous Vej 2, 8000 Aarhus, Denmark

Cultural Data Science

September 30, 2025

Table of contents:

Week 2 - Variables

Week 3 - Loops, Conditionals and Functions

Week 4 - Organizing Data with Pandas

Link to my Github:

https://github.com/Victoriaegeskov/CulturalDataScienceGit

Week 2 - Variables

Author: Johanne Sejrskild
Date: 10.08.2025

Good afternoon
Today we are going to work with variables and text as data. The green excercises will be highly linked to what you did yesterday with Anna. If you find them challenging use Annas powerpoint as a help or ask. If you want to challenge yourself, try and do them all without using any help. In the yellow and red excercises we will be working with text as data. Here we will work with litterary classics and exploring the language used.

Structure of the notebook:
Green excercises

- Variables and storing data in them
- Container variable types
- Accessing elements
- Quiz

Yellow excercises

- Build a lexicon of a word in a book of you choosing
- Compare lengths of the books

Red excercises

- Find the most frequent words
- Compare pronouns used in books written by male and female authors

Start with the first excercise, and then continue in order. Feel free to work together, and see how far you can get.
The important thing is to learn, not to solve all the challenges!

Green excercises

Variables and storing data in them

Fill in the blanks:

Which variable does these data types belong to?

Whole numbers = Integers Decimal numbers = Floats
Collections of characters = Strings True or False values = booelen operators

In [112…
```
# Add some values to the variables below and test if you are correct.
# you can use the type() function to check the data type of a variable.

x = 10
name = "Victoria"
this_is_fun = True
height = 1.70

print(type(x))
print(type(name))
print(type(this_is_fun))
print(type(height))
```

```
<class 'int'>
<class 'str'>
<class 'bool'>
<class 'float'>
```

Container variable types

Python provides several built-in container types, including lists, dictionaries, tuples, and sets. These containers can store collections of data and are essential for effective data manipulation.

Lists are ordered collections of items that can be changed (mutable). Lists are defined by square brackets [].
Dictionaries store key-value pairs, are unordered, and mutable. Defined by curly braces {}.
Tuples are ordered collections of items that cannot be changed (immutable). Defined by parentheses ().
Sets are unordered collections of unique items, mutable but items must be immutable. Defined by curly braces {} or the set() function.

In [47]:
```python
# create a list, dictionary, tuple and a set.

# list
list = ["Albania", "Mexico", "Ireland"]


# dictionary
dict = {"Countries": ["Norway", "Germany", "Denmark"],
        "Capitals": ["Oslo", "Berlin", "Copenhagen"]}

# tuple
tuple = ("Albania", "Mexico", "Ireland")

# set
set1 = {"Norway", "Germany", "Denmark", "Ireland", "Ireland", "Germany", "Germany"}
```

Acessing elements

Access a value from each container variable you just created

In [39]:
```python
# list - Access elements by index, starting at 0.
print(list[2])

# dictionary - Access values by their key.
print(dict['Capitals'])

# tuple - Access elements by index, just like lists.
print(tuple[0])

# set - Cannot access items by index because sets are unordered. but you can check if an item is in the set by usin
print("Spain" in set1)
print("Germany" in set1)
```

```
Ireland
['Oslo', 'Berlin', 'Copenhagen']
Albania
False
True
```

Quiz

1. Which of the following correctly adds a new element “orange” to the fruits list?

fruits.add(orange)
fruits.append("orange")
fruits.insert{"orange"}
fruits += ["orange"]

Answer: fruits.append("orange")

2. How do you add a new key-value pair "gender": "female" to the person dictionary?

person.add("gender" = "female")
person["gender"] = "female"
person.append("gender": "female")
person.insert("gender" = "female")

Answer: person["gender"] = "female"

3. Which of the following statements about tuples is true?

Tuples are mutable.
You can add new elements to a tuple.
You access tuple elements by key.
Tuples are ordered and immutable.

Answer: Tuples are ordered and immutable.

4. What will be the result of adding an existing element to a set?

The set will add another instance of the element.
The set will remain unchanged because it only holds unique elements.
The operation will result in an error.
The set will reorder its elements.

Answer: The set will remain unchanged because it only holds unique elements.

Yellow excercises

Before you start on the *yellow excercises* you have to install and import the package containing all the litterary classics.
You will also get an example of how to build a lexicon of the use of the word "whale" in Moby Dick and the words surrounding it.

So:
Run each cell underneath one at a time, in order. If something in one cell doesn't work right, it might be because you have overwritten a variable, so try going back and running all the previous cells again.

In [52]:
```
# install the natural language toolkit package (nltk), which has a copy of the books.

%pip install nltk
```

```
Requirement already satisfied: nltk in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/python3.13/site-packages (3.9.2)
Requirement already satisfied: click in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/python3.13/site-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/python3.13/site-packages (from nltk) (1.5.1)
Requirement already satisfied: regex>=2021.8.3 in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/python3.13/site-packages (from nltk) (2026.1.15)
Requirement already satisfied: tqdm in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/python3.13/site-packages (from nltk) (4.67.3)
Note: you may need to restart the kernel to use updated packages.
```

In [53]:
```
# import the nltk package so that it is accessible to Python, and download a collection of books from Project Guten
import nltk
nltk.download('gutenberg')
```

```
[nltk_data] Downloading package gutenberg to
[nltk_data]     /Users/victoriaegskovnielsen/nltk_data...
[nltk_data]   Unzipping corpora/gutenberg.zip.
```

Out[53]: True

Now we have installed and imported the books, but we want to extract one and do a bit of data cleeaning so we can look at the words used.
I have chosen to look at Moby Dick so lets find that

In [57]:
```
# Create a variable called "mobydick" which contains the text of the book.
mobydick = nltk.corpus.gutenberg.raw('melville-moby_dick.txt')

# make all characters lowercase
mobydick = mobydick.lower()

# remove the "\n" and "\r" characters, which indicate line breaks in the text (newlines)
mobydick = mobydick.replace('\n', ' ')
mobydick = mobydick.replace('\r', ' ')

# split up the text into a long list of individual words
mobydick = mobydick.split()
```

Question: Why is it necessary to split the text into a list of words in order to count them and maybe do more analysis?

In [56]:
```
# make a list called "lexicon", and fill it with every occurrence of the word "whale", and a few words preceeding a
lexicon = []
for i, val in enumerate(mobydick):
    if val == "whale":
            lexicon.append(str(' '.join(mobydick[i-5:i+5])))
```

In [115…
```
# take a look at what we have found
# lexicon
```

In [59]:
```
# let's see how many instances of the word "whale" were found
len(lexicon)
```

Out[59]: 528

Let's try again, but this time let's just search for "the whale", not "whale" by itself. Try to comment each line of the code and explain what it does

In [ ]:
```
lexicon_twowords = []  # creates empty list to store the results
for i, val in enumerate(mobydick):  # creates a for loop goes through every word in mobydick
                                    # i is the position of the word
                                    # and val refers to the word itself
    if val == "whale":     # find the word "whale" in lexicon
        if mobydick[i-1] == "the":  # find all the places with "the" right in front of whale"
            lexicon_twowords.append(str(' '.join(mobydick[i-5:i+5])))  # get 5 words before and after "the whale" a
```

In [114…
```
# take a look at what we have found
# lexicon_twowords
```

Build a lexicon of a word in a book of you choosing

Now, in the cell below, modify the code from above to search for a word in a book of your own choosing. Give your lexicon a meaningful name that is different from the ones above and comment your code so you understand what is happening each step of the way. But first lets have a look at the books you can choose from.

In [62]:
```
nltk.corpus.gutenberg.fileids()
```

Out[62]:
```
['austen-emma.txt',
 'austen-persuasion.txt',
 'austen-sense.txt',
 'bible-kjv.txt',
 'blake-poems.txt',
 'bryant-stories.txt',
 'burgess-busterbrown.txt',
 'carroll-alice.txt',
 'chesterton-ball.txt',
 'chesterton-brown.txt',
 'chesterton-thursday.txt',
 'edgeworth-parents.txt',
 'melville-moby_dick.txt',
 'milton-paradise.txt',
 'shakespeare-caesar.txt',
 'shakespeare-hamlet.txt',
 'shakespeare-macbeth.txt',
 'whitman-leaves.txt']
```

In [ ]:
```
# Add your code to build your own lexicon here, using the same method as above.

# Create a variable called "hamlet" which contains the text of the book.
hamlet = nltk.corpus.gutenberg.raw('shakespeare-hamlet.txt')

# make all characters lowercase
hamlet = hamlet.lower()

# remove the "\n" and "\r" characters, which indicate line breaks in the text (newlines)
hamlet = hamlet.replace('\n', ' ')
hamlet = hamlet.replace('\r', ' ')

# split up the text into a long list of individual words
hamlet = hamlet.split()



# Create a lexicon looking for the name "ophelia"
lexicon_h = []
for i, val in enumerate(hamlet):
    if val == "ophelia":
            lexicon_h.append(str(' '.join(hamlet[i-5:i+5])))

len(lexicon_h)
```

Out[ ]: 6

Compare lengths of books

We can use the command `len` to find how many items there are in a list. E.g., to find the number of characters in the list called `mobydick`, from earlier, we can write: `len(mobydick)`.

Use the starter code below to find out which book in the books included in `nltk` has the most characters.

In [101…
```
# One way to do it: Print all the titles and numbers of characters
# starter code:

books = nltk.corpus.gutenberg.fileids()

longest_book = ""
most_characters = 0

for title in books:
    book = nltk.corpus.gutenberg.raw(title)
    if len(book) > most_characters:
        longest_book = title
        most_characters = len(book)
print(f"This is the longest book: {longest_book} with {most_characters} characters")


    # Maybe an if statement here checkin if the current book is longer than most_characters replace the title with
```

```
This is the longest book: bible-kjv.txt with 4332554 characters
```

Optional more advanced way to do it, for those with python experience up for a challenge.

Here we have to find the amount of words not characters. So we need to use `len(book.split())` to split the characters up to words

In [102…
```python
# Another way to do it: Make a list of titles and a list of wordcounts, put it in a dataframe, then sort them based
%pip install pandas
import pandas as pd

# starter code:
books = nltk.corpus.gutenberg.fileids()

titles = []
numwords = []

for title in books:
    book = nltk.corpus.gutenberg.raw(title)
    len(book.split())
    # Add title and length of book

# add titles and numwords to a df
# Sort them

#Print the head of the dataframe
```

```
Requirement already satisfied: pandas in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/python3.13/s
ite-packages (2.3.2)
Requirement already satisfied: numpy>=1.26.0 in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/pytho
n3.13/site-packages (from pandas) (2.3.1)
Requirement already satisfied: python-dateutil>=2.8.2 in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/
lib/python3.13/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/python
3.13/site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/pyth
on3.13/site-packages (from pandas) (2025.2)
Requirement already satisfied: six>=1.5 in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/python3.1
3/site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

Red excercises

Find the most frequent words in the book you choose earlier

`nltk` has a built-in function called `FreqDist` which counts up how many times each word in a text occurs. So, if you have a list called `words` which contains all the words in a book, you can find the frequencies of all of them by writing `freq = nltk.FreqDist(words)`. You can then get the ten most common words by writing `freq.most_common(10)` and so on.
What are the ten most common words in your book?

In [109…
```python
# starter code:

# Create a variable called "hamlet" which contains the text of the book.
hamlet = nltk.corpus.gutenberg.raw('shakespeare-hamlet.txt')

# make all characters lowercase
hamlet = hamlet.lower()

# remove the "\n" and "\r" characters, which indicate line breaks in the text (newlines)
hamlet = hamlet.replace('\n', ' ')
hamlet = hamlet.replace('\r', ' ')

# split up the text into a long list of individual words
hamlet = hamlet.split()



# Use frequcy distibution and find most commen 10 word
freq = nltk.FreqDist(hamlet)
print(freq.most_common(10))



# Replace new lines \n with empty space
# Replace \r with empty space
# Split the characters into words
# use the function FreqDist
# Print the 10 most common words
```

```
[('the', 992), ('and', 860), ('to', 683), ('of', 605), ('i', 520), ('my', 499), ('a', 497), ('you', 449), ('in', 37
7), ('it', 349)]
```

You might not feel like the ten most frequent words are of any real value in a potential project for analysis or comparison. This is due to the fact that the most frequent words in our language is what we call *stopwords*. These words like "a" and "the" are so common in

English, that they don't really tell us much about the text.

That is why we often remove "stopwords", that is, a list of the most common words in English, before e.g. counting frequencies.

There are several of these lists available, in English) as well as other languages, such as Danish.

Below is some starter code to remove stopwords. Use these snippets to see what the most common words in your book is after removing these most common words.

In [ ]:
```
# list of stopwords

stopwords = ["", "i", "me", "my", "myself", "we", "our", "ours", "ourselves", "you", "your", "yours", "yourself", "

# code to remove stopwords.
# words = [word for word in words if word not in stopwords]
```

In [111…
```
# starter code:

#book = nltk.corpus.gutenberg.raw('yourbook.txt')
#words = book.lower()

# Remove stopwords in text
hamlet = [word for word in hamlet if word not in stopwords]


# Use frequcy distibution and find most commen 10 word
freq = nltk.FreqDist(hamlet)
print(freq.most_common(10))
```

```
[('ham.', 337), ('haue', 171), ('shall', 106), ('thou', 101), ('king.', 96), ('hor.', 95), ('let', 94), ('good', 9
1), ('thy', 90), ('lord', 83)]
```

Compare the pronouns used in books written by male and female authors

This could be an example of a project
One could imagine that authors draw most inspiration from there own life and therefore write about characters resembeling themselves. Investigate whether male and femate authors of novels write more pronouns they identify with themselves.

Is there any evidence for this postulation in our data?

If you can, it would be neat to make a loop that goes through all books, and creates a dataframe that you afterwards can sort in to remove non-novels :)

In [ ]:
```
# Example of structure:

# Make a set of male and female pronouns (2 sets)
# Make a dictionary of all titles in the corpus and the authors gender (1 dictionary)
# Make empty lists to store title, genders, number of female pronouns and number of male pronouns
# Make a loop that iterates over each book, splits the text to words, counts the number of pronouns (hint: the code
# Make dataframe, fill it with the lists above, and print it
```

Discuss:
What is your findings and what is the limitations of this project

Week 3 - Loops, Conditions and Functions

Author: Johanne Sejrskild
Date: 12.08.2025

Good afternoon
Today we are going to work with loops, condions and functions. The green excercises will be highly linked to what you livecoded with Anna. If you find them challenging use yesterdays work as a help or ask. If you want to challenge yourself, try and do them all without using any help. In the yellow excercises we will make a password checker and a function that converts temperatures. And for the red excercise we will make a function that calculates grades

Structure of the notebook:
Green excercises

- Conditional Statements
- Loops
- Functions

Yellow excercises

- Password checker
- Temperature converter

Red excercises

- Grade calculator
- Optional extra: Grade calculator as a class

Start with the first excercise, and then continue in order. Feel free to work together, and see how far you can get.
The important thing is to learn, not to solve all the challenges!

In [1]:
```
# Before we start we need to import the necessary packages
%pip install pandas
%pip install lxml
import pandas as pd
```

```
Requirement already satisfied: pandas in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/python3.13/s
ite-packages (2.3.2)
Requirement already satisfied: numpy>=1.26.0 in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/pytho
n3.13/site-packages (from pandas) (2.3.1)
Requirement already satisfied: python-dateutil>=2.8.2 in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/
lib/python3.13/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/python
3.13/site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/pyth
on3.13/site-packages (from pandas) (2025.2)
Requirement already satisfied: six>=1.5 in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/python3.1
3/site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
Collecting lxml
  Downloading lxml-6.1.3-cp313-cp313-macosx_10_13_universal2.whl.metadata (3.3 kB)
Downloading lxml-6.1.3-cp313-cp313-macosx_10_13_universal2.whl (8.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.6/8.6 MB 30.4 MB/s  0:00:00 eta 0:00:01
Installing collected packages: lxml
Successfully installed lxml-6.1.3
Note: you may need to restart the kernel to use updated packages.
```

Green excercises

Conditional Statements

The *if* statement

In [ ]:
```
# Basic syntax
# if condition:
    # code to execute if condition is True
```

In [4]:
```
# Example of if statement
# Before running: What will happen and why?

x = 10
if x > 5:
    print("x is greater than 5")
```

```
x is greater than 5
```

In [5]:
```
# Exceise: Modify the code to check if x is less than 5 and print a different message.
#
x = 4
if x < 5:
    print("x is lower than 5")
```

```
x is lower than 5
```

The *else* statement

In [ ]:
```
# Basic syntax

# if condition:
    # code if condition is True
# else:
    # code if condition is False
```

In [ ]:
```
# Example of if-else statement
# Before running: What will happen and why?

x = 3
if x > 5:
    print("x is greater than 5")
else:
    print("x is not greater than 5")
```

In [ ]:
```
# Excercise - Change the code above so that it prints "x is greater than 5." by only changing one number/symbol.
# How many solutions can you find? and is some of them better than others?

x = 6
if x > 5:
    print("x is greater than 5")
else:
    print("x is not greater than 5")


# Answer:
# Writing x = 6 or 7 or 8 or etc.
```

```
x is greater than 5
```

The *elif* statement (for multiple conditions)

In [ ]:
```
# Basic syntax

# if condition1:
    # code if condition1 is True
# elif condition2:
    # code if condition2 is True
# else:
    # code if neither condition is True
```

In [ ]:
```
# Example of if-elif-else statement
# Before running: What will happen and why?

x = 5
if x < 5:
    print("x is greater than 5")
elif x == 5:
    print("x is equal to 5")
else:
    print("x is less than 5")

# Answer:
# It will print "x is equeal to 5" because x is 5.
```

```
x is equal to 5
```

In [ ]:
```
## Exercise - What happens when the condition for if and elif are the same? Why?

# Answer:
#  Then it goes for the if statements first - as elif means "else if"
```

Loops

Loops in Python are used to execute a block of code repeatedly. We will look at two different types of loops; the *for loop*, and the *while loop*.

The for loop iterates over sequences, so it keeps executing until it has reached the end of the loop.

The while loop keeps repeating while a condition is true, and first ends when the condition becomes false.

In [13]:
```python
# For loop - Basic syntax
# for element in sequence:
    # code to execute for each element

# Example of for loop
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

```
apple
banana
cherry
```

In [40]:
```python
# Exercise - Modify the code to print the length of each fruit instead of the fruit itself.

fruits = ["apple", "banana", "cherry"]

for word in fruits:
    print('There are', len(word), 'letters in', word.upper())
```

```
There are 5 letters in APPLE
There are 6 letters in BANANA
There are 6 letters in CHERRY
```

In [14]:
```python
# While loop - Basic syntax
# while condition:
    # code to execute while condition is True

# Example of while loop
x = 5
while x > 0:
    print(x)
    x -= 1
```

```
5
4
3
2
1
```

In [60]:
```python
# Excercise - Modify the code to count up from 1 to 5 instead of counting down from 5 to 1.

x = 1
while x < 6:
    print(x)
    x += 1
```

```
1
2
3
4
5
```

Functions

In [36]:
```python
# Heres a calculator
def add_two_numbers(a, b):
    result = a + b
    print(f"{a} + {b} = {result}")
```

In [37]:
```python
# Try it out with different numbers
add_two_numbers(6, 9)
add_two_numbers(10, 11)
```

```
6 + 9 = 15
10 + 11 = 21
```

In [38]:
```python
# Make a subtraction function
def sub_two_numbers(a, b):
    result = a - b
    print(f"{a} - {b} = {result}")

sub_two_numbers(6, 9)
sub_two_numbers(10, 11)
```

```
6 - 9 = -3
10 - 11 = -1
```

Yellow excercises

Password checker

You're making a simple password checker. How would you check if the password is correct?

Write code that asks for a password and checks if it follows the requirements:

- It needs to be between 8 and 32 characters long

Make sure you print some meaningful message the user can understand. Try it with both the correct password and wrong passwords.

Hint: use the input function to allow for user inputs e.g. **input("Enter the password: ")**

In [88]:
```
# code here

def password_checker():
    password = input("Write your password:")
    digits = 0
    if len(password) <= 8 or len(password)>= 32:
        print(f"The password has to be between 8 & 32 digits")
    else:
        print("This is a great password")

password_checker()
```

```
The password has to be between 8 & 32 digits
```

Now lets add some extra requirements:

- The password needs to have at least 3 digits
- And there needs to be an uppercase letter

In [104…
```
# Modified code here

def password_checker2():
    password = input("Write your password:")
    digits = 0
    uppercase = 0

    for charector in password:
        if charector.isdigit():
            digits = digits +1
        if charector.isupper():
            uppercase =+1

    if len(password) <= 8 or len(password)>= 32:
        print("The password does not meet the requirements")
    elif digits < 3:
        print("You need at least 3 numbers in your password")
    elif uppercase == 0:
        print("You forgot an uppercase charecter")

    else:
        print("This is a great password")


password_checker2()
```

```
You need at least 3 numbers in your password
```

And the last bits, now we also need to ensure that the password contains:

- 2 special characters

Make sure the user understands why a wrong password is rejected. And ensure that only a password that holds all requirements are passed.

In [ ]:
```
# Last modifications on the code - final password_checker


def password_checker3():
    password = input("Write your password:") # Makes input column in searchbar above
    digits = 0      # A kind of counting variable starting at zero
    uppercase = 0
    special_characters = "!@#$%^&*()-+?_=,<>/" # Defining which charectors that will count for special
    special_count = 0


    for charector in password:
        if charector.isdigit():
            digits = digits +1
        if charector.isupper():
            uppercase +=1
        if charector in special_characters:
            special_count +=1
    # This parts checks if charector is digits, upper or a special charector and then add it to the empty appropria

    if len(password) <= 8 or len(password)>= 32:
        print("The password does not meet the requirements")
    elif digits < 3:
```

```
        print("You need at least 3 numbers in your password")
    elif uppercase == 0:
        print("You forgot an uppercase charecter")
    elif special_count < 2:
        print("You need two special charecters like: !@#$%^&*()-+?_=,<>/")


    else:
        print("This is a great password")


password_checker3()
```

```
This is a great password
```

Temperature converter

Lets now make a function that can convert celcius to fahrenheit. Make an IPO with your group and discuss what hte input, the process and the outpus should be.

The function should print an informational message and return something usable.

Hint: F = (celsius * 9/5) +32

In [269…

```
# celsius to fahrenheit converter
def fahrenheit_converter(x):
    result = (x * 1.8) + 32
    print(f"{x} in celsius is {result} in fahrenheit")

fahrenheit_converter(25)
```

```
25 in celsius is 77.0 in fahrenheit
```

Now make a function that does the opposite.
Hint: C = ((fahrenheit-32)/(9/5))

In [268…

```
# Fahrenheit to Celsius converter
def celsius_converter(y):
    result =  (y - 32) / 1.8
    print(f"{y} in fahrenheit is {result} in celsius")

celsius_converter(77)
```

```
77 in fahrenheit is 25.0 in celsius
```

Well done! Now wouldn't it be neat if we could just wrap that into a function that we could call temperature_converter with two parameters "temperature" and "unit" so that we could convert both ways.

In [ ]:

```
# Temperature converter here

def temperature_converter(temperature, unit):
    if unit == "fahrenheit":
        result = (temperature - 32) / 1.8
        print(f"{temperature} in fahrenheit is {result} in celsius")
    elif unit == "celsius":
        result = (temperature * 1.8) + 32
        print(f"{temperature} in celsius is {result} in fahrenheit")
    else:
        print("Please enter correct unit - either fahrenheit or celsius")



temperature_converter(25, "celsius")

temperature_converter(77, "fahrenheit")
```

```
25 in celsius is 77.0 in fahrenheit
77 in fahrenheit is 25.0 in celsius
```

OBS: To ensure you can go back in 3 months time and read you code and understand the logics behind it it needs to be well commented.
So, take look at the yellow excercises and see if you need to add some meaningful comments about the logics and coding choices.

:))

Red excercises

Grade calculator

The scenario: You're helping a teacher who wants to calculate student grades quickly. The teacher has been doing these calculations by hand, which takes a long time and sometimes leads to mistakes.

The challenge:

- Calculate the average of test scores
- Convert percentage grades to letter grades (it's just easier than the danish grading metric ngl)
- Calculate final grades with different weights for different assignments

Example data:
Tests count for 70%, homework counts for 30%

Test scores: 85, 93, 78, 96, 88 Homework average: 90

Now we start with one thing first. Each functions should do *one* thing effectively.

In [143…
```python
# Make a function that calculate the average of test scores.
def calculate_average(*testscores):
    average =  sum(testscores)/ len(testscores)
    print(f"The testscore average is {average}")

calculate_average(85, 93, 78, 96, 88, 90, 87)
```

```
The testscore average is 88.14285714285714
```

Well done, now we move on to the next sub-task

Hint: Grade to lettter grade converter
90–100 → A
80–89 → B
70–79 → C
60–69 → D
0–59 → F

In [ ]:
```python
# Make a function that converst the test scores to letter grades

def percentage_to_letter_grade(grade):
    if grade >= 90:
        print(f"{grade} gives an A")
    elif grade >= 80:
            print(f"{grade} gives an B")
    elif grade >= 70:
        print(f"{grade} gives an C")
    elif grade >= 60:
        print(f"{grade} gives an D")
    else:
        print(f"{grade} gives an F")

# this function exploit the order of "if", "elif". If the first is not true, then it moves on to the next and so on
# This makes us able to avoid having to define e.g. "grade >= 70 and >=79"


# Test with different percentages
percentage_to_letter_grade(79)
percentage_to_letter_grade(91)
percentage_to_letter_grade(25)
percentage_to_letter_grade(81)
percentage_to_letter_grade(65)
```

```
79 gives an C
91 gives an A
25 gives an F
81 gives an B
65 gives an D
```

Last task, We need to calculate the weighted grade

In [197…
```python
def calculate_weighted_grade(*testscores, homework):
    testscore_weighted_average =  (sum(testscores)/ len(testscores)) *0.7
    homework_weigted_average = (sum(homework)/ len(homework)) * 0.3
    final_weighted_average =  testscore_weighted_average + homework_weigted_average
    print(f"The final score is {final_weighted_average}")



calculate_weighted_grade(85, 78, 77, 95, 100, 45, homework= (53, 60, 76))
```

```
The final score is 74.9
```

Now we want to give the professor one function that uses all these three and wraps them in a neat little function with everything put together.

In [200…
```python
def calculate_student_grade(*testscores, homework):
    testscore_average =  sum(testscores)/ len(testscores)
    homework_average =  sum(homework)/ len(homework)

    print(f"The testscore average is {testscore_average} and homework average is {homework_average}")

    testscore_weighted_average =  testscore_average *0.7
    homework_weigted_average = homework_average * 0.3
    final_weighted_average =  testscore_weighted_average + homework_weigted_average
    print(f"The final score is {final_weighted_average}")

    if final_weighted_average >= 90:
        print(f"{final_weighted_average} gives an A")
    elif final_weighted_average >= 80:
            print(f"{final_weighted_average} gives an B")
    elif final_weighted_average >= 70:
        print(f"{final_weighted_average} gives an C")
    elif final_weighted_average >= 60:
        print(f"{final_weighted_average} gives an D")
    else:
        print(f"{final_weighted_average} gives an F")


calculate_student_grade(85, 78, 77, 95, 100, 45, homework=(50,39, 76))
```

```
The testscore average is 80.0 and homework average is 55.0
The final score is 72.5
72.5 gives an C
```

Optional red: Take your grade calculator and make it a class!

Wuhuu - now you are all done. Impressive!

If you have time alter the function above so it outputs a neat dtudent grade report with test scores, test average, homework average, final grade and letter grade.

In [ ]:
```python
# Code here :)
class GradeCalculator:
    def __init__(self, testscores, homework):
        self.testscores = testscores
        self.homework = homework


    def StartInfo(self):
        print("    STUDENT GRADE REPORT:")
        print(f"The testscores are {self.testscores} ")
        print(f"The homework scores are {self.homework} ")


    def Averages(self):
        self.testscore_average =  sum(self.testscores)/ len(self.testscores)
        self.homework_average =  sum(self.homework)/ len(self.homework)
        print(f"The testscore average is {self.testscore_average}")
        print(f"The homework average is {self.homework_average}")

    def WeightedAverages(self):
        self.testscore_weigthed_average =  self.testscore_average * 0.7  # 70% weight
        self.homework_weigthed_average = self.homework_average * 0.3  # 30 % weight
        self.final_weigthed_average =  self.testscore_weigthed_average + self.homework_weigthed_average
        print(f"The final grade is {self.final_weigthed_average}")


    def LetterGrade(self):
        if self.final_weigthed_average >= 90:
            print(f"The final grade {self.final_weigthed_average} gives grade A")
        elif self.final_weigthed_average >= 80:
            print(f"The final grade {self.final_weigthed_average} gives grade B")
        elif self.final_weigthed_average >= 70:
            print(f"The final grade {self.final_weigthed_average} gives grade C")
        elif self.final_weigthed_average >= 60:
            print(f"The final grade {self.final_weigthed_average} gives grade D")
        else:
            print(f"The final grade {self.final_weigthed_average} gives grade F")

    # Definig method/function that runs everything
    def CalculateGrade(self):
        self.StartInfo()
        self.Averages()
        self.WeightedAverages()
        self.LetterGrade()
```

```python
# Creates some different instanses
Student1 = GradeCalculator([85, 78, 77, 95, 100, 69],[50, 65, 77])
Student2 = GradeCalculator([56, 66, 76, 80, 65, 68],[62, 63, 79])
Student3 = GradeCalculator([90, 94, 92, 95, 100, 90],[88, 87, 65])



# Two ways of accesing/using the GradeCalculator
Student1.StartInfo()
Student1.Averages()
Student1.WeightedAverages()
Student1.LetterGrade()

# Or

Student2.CalculateGrade()
Student3.CalculateGrade()
```

```
    STUDENT GRADE REPORT:
The testscores are [85, 78, 77, 95, 100, 69]
The homework scores are [50, 65, 77]
The testscore average is 84.0
The homework average is 64.0
The final grade is 78.0
The final grade 78.0 gives grade C
    STUDENT GRADE REPORT:
The testscores are [56, 66, 76, 80, 65, 68]
The homework scores are [62, 63, 79]
The testscore average is 68.5
The homework average is 68.0
The final grade is 68.35
The final grade 68.35 gives grade D
    STUDENT GRADE REPORT:
The testscores are [90, 94, 92, 95, 100, 90]
The homework scores are [88, 87, 65]
The testscore average is 93.5
The homework average is 80.0
The final grade is 89.45
The final grade 89.45 gives grade B
```

Week 4 - Data organising with Pandas

Author: Johanne Sejrskild
Date: 22.09.2025

Good afternoon
Today we are going to work with loops, condions and using ´pandas´to manipulate data. The green excercises will be highly linked to what you livecoded with Anna. If you find them challenging use yesterdays work as a help or ask. If you want to challenge yourself, try and do them all without using any help. In the yellow excercises we will do some data manipulation challenges using pandas. And we will skip the red tasks today as we have a lot on the program

Structure of the notebook:
Green excercises

- Data wrangling on the iris dataset

Yellow excercises

- Music sales challenge
- Space mission challenge
- Supervillan challenge

Start with the first excercise, and then continue in order. Feel free to work together, and see how far you can get.
The important thing is to learn, not to solve all the challenges!

In [2]:
```
# Before we start we need to import the necessary packages - ater you have run this chunk you can comment out the f
%pip install pandas
%pip install lxml
%pip install scikit-learn

import pandas as pd
import requests # We might need this package to get some data from the web
from sklearn import datasets
from io import StringIO
```

```
Requirement already satisfied: pandas in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/python3.13/s
ite-packages (2.3.2)
Requirement already satisfied: numpy>=1.26.0 in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/pytho
n3.13/site-packages (from pandas) (2.3.1)
Requirement already satisfied: python-dateutil>=2.8.2 in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/
lib/python3.13/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/python
3.13/site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/pyth
on3.13/site-packages (from pandas) (2025.2)
Requirement already satisfied: six>=1.5 in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/python3.1
3/site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
Requirement already satisfied: lxml in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/python3.13/sit
e-packages (6.1.3)
Note: you may need to restart the kernel to use updated packages.
Requirement already satisfied: scikit-learn in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/python
3.13/site-packages (1.7.1)
Requirement already satisfied: numpy>=1.22.0 in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/pytho
n3.13/site-packages (from scikit-learn) (2.3.1)
Requirement already satisfied: scipy>=1.8.0 in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/python
3.13/site-packages (from scikit-learn) (1.16.1)
Requirement already satisfied: joblib>=1.2.0 in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/lib/pytho
n3.13/site-packages (from scikit-learn) (1.5.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in /Users/victoriaegskovnielsen/miniconda3/envs/methods3_2025/li
b/python3.13/site-packages (from scikit-learn) (3.5.0)
Note: you may need to restart the kernel to use updated packages.
```

Green excercises

Data organisation using a dataset about flowers

In [3]:
```
flower = datasets.load_iris()

# convert to DataFrame
df = pd.DataFrame(flower.data, columns=flower.feature_names)

df.head()
```

Out[3]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Lets take a look at the data frame

In [4]:
```
# There are some commands in the library pandas that can give you a quick overview of the data :)

df.head()      # first 5 rows, if you put a number into the paranthesis you can decide how many rows
df.tail()      # last 5 rows
df.info()      # summary of columns and types
df.describe()  # quick statistics (for numbers)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 4 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
dtypes: float64(4)
memory usage: 4.8 KB
```

Out[4]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Selecting columns and rows

Try to run the cell below and figure out which output is linked to the code

In [5]:
```
# If you want to select a specific column you can select it using the name:
print(df['sepal length (cm)'].head())

# If you would like to print one row, you can use the index of the row:
print(df.iloc[0])

# if you want to select a few rows of only a few columns you can also use indexing:
print(df.iloc[0:3 , 0:2])  # first three rows, first two columns

# And if you want to select specific data, you can specify a single row and column:
print(df.iloc[2,0])  # second row, first column

# Or use the column name:
print(df.loc[2, "sepal length (cm)"]  )
```

```
0    5.1
1    4.9
2    4.7
3    4.6
4    5.0
Name: sepal length (cm), dtype: float64
sepal length (cm)    5.1
sepal width (cm)     3.5
petal length (cm)    1.4
petal width (cm)     0.2
Name: 0, dtype: float64
   sepal length (cm)  sepal width (cm)
0                5.1               3.5
1                4.9               3.0
2                4.7               3.2
4.7
4.7
```

Subsetting data

Subsetting is the process of retrieving just the parts of large files which are of interest for a specific purpose.

This will come in handy for your projects when you have to work with potentially large data files

In [6]:
```
# Let's try to select some data using conditionals

# Here we select all rows where the sepal length is larger than or equal to 5 cm
lengt_above_five = df[df["sepal length (cm)"] >= 5]
lengt_above_five.head()
```

Out[6]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
4	5.0	3.6	1.4	0.2
5	5.4	3.9	1.7	0.4
7	5.0	3.4	1.5	0.2
10	5.4	3.7	1.5	0.2

In [7]:
```
# Here we select all rows where the sepal length is larger than or equal to 5 cm and the sepal width is less than 2
length_and_width = df[(df["sepal length (cm)"] >= 5) & (df["sepal width (cm)"] < 3.5)]
length_and_width.head()
```

Out[7]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
7	5.0	3.4	1.5	0.2
20	5.4	3.4	1.7	0.2
23	5.1	3.3	1.7	0.5
25	5.0	3.0	1.6	0.2
26	5.0	3.4	1.6	0.4

In [ ]:
```
# Excercise - Find the longest petal length and the median petal length
# and subset the flowers that are between the median and one centimeter shorter than max length
# If you want to select a specific column you can select it using the name:



# The longest petal length & The median petal length
petal_max = df["petal length (cm)"].max()
petal_median = df["petal length (cm)"].median()

# Subset of flowers
long_petals = df[(df["petal length (cm)"] >= petal_median) & (df["petal length (cm)"] <= petal_max - 1)]

long_petals.head()
```

Out[ ]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
50	7.0	3.2	4.7	1.4
51	6.4	3.2	4.5	1.5
52	6.9	3.1	4.9	1.5
54	6.5	2.8	4.6	1.5
55	5.7	2.8	4.5	1.3

Sorting data

We can choose to sort our data in order of something of interest.

In [9]:
```
# we could sort the data by a specific column in both ascending and descending order
df_sorted = df.sort_values(by="sepal length (cm)", ascending=False) # change direction by True/False so if you want
df_sorted.head()
```

Out[9]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
131	7.9	3.8	6.4	2.0
135	7.7	3.0	6.1	2.3
122	7.7	2.8	6.7	2.0
117	7.7	3.8	6.7	2.2
118	7.7	2.6	6.9	2.3

In [ ]:
```
# Excercise - sort the data by petal width in ascending order and select the 10 flowers with the smallest petal wid
```

```
df_sorted1 = df.sort_values(by="petal width (cm)", ascending=True)
# df_sorted1


smallest_pedal_width = df_sorted1.iloc[0:10, :]

smallest_pedal_width
```

Out[ ]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
32	5.2	4.1	1.5	0.1
13	4.3	3.0	1.1	0.1
37	4.9	3.6	1.4	0.1
9	4.9	3.1	1.5	0.1
12	4.8	3.0	1.4	0.1
0	5.1	3.5	1.4	0.2
27	5.2	3.5	1.5	0.2
28	5.2	3.4	1.4	0.2
29	4.7	3.2	1.6	0.2
30	4.8	3.1	1.6	0.2

Flipping

Should you work with time seires data and would like to mirror (flip) your data, you can do this using pandas

In [11]:
```
print(df.head(5))

reversed_df = df.iloc[::-1]   # Flipping the dataframe horisontally (reverse rows)

print(reversed_df.head(5))
```

```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2
     sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
149                5.9               3.0                5.1               1.8
148                6.2               3.4                5.4               2.3
147                6.5               3.0                5.2               2.0
146                6.3               2.5                5.0               1.9
145                6.7               3.0                5.2               2.3
```

Joining Sometimes we have multiple dataframes we woudl like to add together. Maybe you have been subsetting parts of an old dataframe to substract important information and would now like join them so you can begin your analysis.

In [12]:
```
# Joining a bit of the iris data with a new dataframe (we will make up some data here)
first_10 = df.iloc[0:10, :]  # selecting the first 10 rows of the iris data
new_data = {"color": ["red", "blue", "green", "yellow", "purple", "red", "blue", "green", "yellow", "purple"],
            "height": [80, 80, 70, 100, 90, 80, 80, 70, 100, 90]}
# Right now new_df is a dictionary, we need to convert it to a dataframe
new_df = pd.DataFrame(new_data)

#Now we join the two dataframes
joined_left = first_10.join(new_df, how='left') # There are 4 different types of how: outer, inner, left, right.

joined_left
```

Out[12]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	color	height
0	5.1	3.5	1.4	0.2	red	80
1	4.9	3.0	1.4	0.2	blue	80
2	4.7	3.2	1.3	0.2	green	70
3	4.6	3.1	1.5	0.2	yellow	100
4	5.0	3.6	1.4	0.2	purple	90
5	5.4	3.9	1.7	0.4	red	80
6	4.6	3.4	1.4	0.3	blue	80
7	5.0	3.4	1.5	0.2	green	70
8	4.4	2.9	1.4	0.2	yellow	100
9	4.9	3.1	1.5	0.1	purple	90

Different types of how to join two data frames
This is important if your dataframes do not have the same amount of rows

left → all rows from the left DataFrame - and the ones matching from the right.

right → all rows from the right DataFrame - and the ones matching from the right.

inner → only rows with matching index values (rows) in both.

outer → all rows from both, fill missing with NaN.

In [13]:
```
# Excercise - Which types of join (the 'how=') will work in the example above? Try them out and see what happens
joined_right = first_10.join(new_df, how='right')

joined_inner = first_10.join(new_df, how='inner')

joined_outer = first_10.join(new_df, how='outer')
```

In [ ]:
```
# Excercise 2 - Add a row to one of the dataframes and see what happens when you join them again

first_11 = df.iloc[0:11, :]  # selecting the first 11 rows of the iris data


joined_left = first_11.join(new_df, how='left')
#joined_left


joined_right = first_11.join(new_df, how='right')
#joined_right


joined_inner = first_11.join(new_df, how='inner')
#joined_inner

joined_outer = first_11.join(new_df, how='outer')
joined_outer
```

Out[ ]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	color	height
0	5.1	3.5	1.4	0.2	red	80.0
1	4.9	3.0	1.4	0.2	blue	80.0
2	4.7	3.2	1.3	0.2	green	70.0
3	4.6	3.1	1.5	0.2	yellow	100.0
4	5.0	3.6	1.4	0.2	purple	90.0
5	5.4	3.9	1.7	0.4	red	80.0
6	4.6	3.4	1.4	0.3	blue	80.0
7	5.0	3.4	1.5	0.2	green	70.0
8	4.4	2.9	1.4	0.2	yellow	100.0
9	4.9	3.1	1.5	0.1	purple	90.0
10	5.4	3.7	1.5	0.2	NaN	NaN

Concatenating

You can also join two dataframes bu simply gluing them together.

In [15]:
```
# We just made a subset of the original dataframe called 'first_10' now we find the last 10 and glue them together
last_10 = df.iloc[-10:, :]   # selecting the last 10 rows using one of the methods we learned above


# Now we concatenate the two dataframes together
concatenated = pd.concat( [first_10, last_10], axis=0)  # axis=0 means we concatenate rows, axis=1 would concatenat
concatenated
```

Out[15]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
5	5.4	3.9	1.7	0.4
6	4.6	3.4	1.4	0.3
7	5.0	3.4	1.5	0.2
8	4.4	2.9	1.4	0.2
9	4.9	3.1	1.5	0.1
140	6.7	3.1	5.6	2.4
141	6.9	3.1	5.1	2.3
142	5.8	2.7	5.1	1.9
143	6.8	3.2	5.9	2.3
144	6.7	3.3	5.7	2.5
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

Now you have played around with some of the basics manipulation in pandas! Now lets jump into some challenges

Yellow excercises

OBS: To ensure you can go back in 3 months time and read you code and understand the logics behind it it needs to be well commented.
So, while you solve the yellow excercises ensure that you add some meaningful comments about the logics and coding choices.

:))

The Yellow excercises is borrowed from the 2024 couse written by Ethan Weed

Music sales challenge

Write a script that:

1. Combines the tables of best-selling physical singles and best-selling digital singles on the Wikipedia page "List_of_best-selling_singles"
2. Outputs the artist and single name for the year you were born. If there is no entry for that year, take the closest year after you were born.
3. Outputs the artist and single name for the year you were 15 years old.

In [16]:
```
# Starter code
url_music = "https://en.wikipedia.org/wiki/List_of_best-selling_singles"

# Add a User-Agent header so Wikipedia doesn't block it
headers = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"}
response = requests.get(url_music, headers=headers)

# Pass the HTML text to pandas
musicdata = pd.read_html(StringIO(response.text))

#Extracting physical and digital singles from the musicdata
physical_singles = musicdata[0]
digital_singles = musicdata[3]

physical_singles["Type"] = "Physical"
digital_singles["Type"] = "Digital"

# Combining the two tables
combined_singles = pd.concat(
    [physical_singles, digital_singles],
    ignore_index=True
)
```

```
#print(combined_singles.head())
combined_singles.head()
```

Out[16]:

	Year	Single	Artist	Claimed sales (in million)	Ref.	Type	Units (in million)
0	1935.0	"Silent Night"	Bing Crosby	30.0	[11]	Physical	NaN
1	1942.0	"White Christmas"	Bing Crosby	50.0	[5]	Physical	NaN
2	1946.0	"Petit Papa Noël"	Tino Rossi	30.0	[12]	Physical	NaN
3	1954.0	"Rock Around the Clock"	Bill Haley & His Comets	25.0	[13][14]	Physical	NaN
4	1960.0	"It's Now or Never"	Elvis Presley	20.0	[15][16]	Physical	NaN

In [103…

```
# Print the artist and single from the year you were born

#Hint: combined_singles[combined_singles['Year']] finds the column with all the years songs were released

singles_2002 = combined_singles[combined_singles['Year'] == 2010.0]
print(singles_2002[["Single", "Artist"]])
```

```
                Single Artist
8  "Rolling in the Deep"  Adele
```

In [104…

```
# Print the artist and single from the year you were 15 years old.

singles_2017 = combined_singles[combined_singles['Year'] == 2017.0]
print(singles_2017[["Single", "Artist"]])
```

```
           Single      Artist
15  "Shape of You"  Ed Sheeran
```

Space challenge

1. Make a single dataframe that combines the space missions from the 1950's to the 2020's
2. Write a script that returns the year with the most launches
3. Write a script that returns the most common month for launches
4. Write a script that ranks the months from most launches to fewest launches

In [19]:

```
# Starter code.
url_space =  "https://en.wikipedia.org/wiki/Timeline_of_Solar_System_exploration"

# Add a User-Agent header so Wikipedia doesn't block it
headers = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"}
response = requests.get(url_space, headers=headers)

# Pass the HTML text to pandas
spacedata = pd.read_html(StringIO(response.text))

# combine all tables into data frame
combined_space = pd.concat(spacedata, ignore_index = True)

# Dropping column we dont need
combined_space = combined_space.iloc[:, 0:3]
combined_space.head(20)
```

Out[19]:

	Mission name	Launch date	Description
0	Sputnik 1	4 October 1957	First Earth orbiter
1	Sputnik 2	3 November 1957	Earth orbiter, first animal in orbit, a dog na...
2	Explorer 1	1 February 1958	Earth orbiter; discovered Van Allen radiation ...
3	Vanguard 1	17 March 1958	Earth orbiter; oldest spacecraft still in Eart...
4	Luna 1	2 January 1959	First lunar flyby (failed lunar impact); first...
5	Pioneer 4	3 March 1959	Lunar flyby
6	Luna 2	12 September 1959	First extraterrestrial impact and lunar impact...
7	Luna 3	4 October 1959	Lunar flyby; First images of another celestial...
8	Pioneer 5	11 March 1960	Interplanetary space investigations
9	Venera 1	12 February 1961	First probe to another planet; Venus flyby (co...
10	Vostok 1	12 April 1961	First crewed Earth orbiter (Yuri Gagarin)
11	Ranger 1	23 August 1961	Attempted lunar test flight (failed to leave E...
12	Ranger 2	18 November 1961	Attempted lunar test flight (failed to leave E...
13	Ranger 3	26 January 1962	Attempted lunar impact (missed Moon)
14	Ranger 4	23 April 1962	Lunar impact (but unintentionally became the f...
15	Mariner 2	27 August 1962	First successful planetary encounter, First su...
16	Ranger 5	18 October 1962	Attempted lunar impact (missed Moon)
17	Mars 1	1 November 1962	First probe to Mars: flyby (contact lost)
18	Luna 4	2 April 1963	Attempted lunar lander (missed Moon)
19	Cosmos 21	11 November 1963	Attempted Venera test flight?

In [ ]:
```
## The year with the most launches

#max_year = combined_space["Launch date"].astype(str)
launch_year = combined_space["Launch date"].str[-4:]

print(launch_year.value_counts().head())


# Hint:  we want the column 'Launch date', only the rows with a date, and then converteed to a string
# Because a string we can index so we can take the last four digits
```

```
Launch date
1965    12
1967    12
1966    11
2025    11
1969    10
Name: count, dtype: int64
```

In [68]:
```
# The month with the most launches
launch_month = combined_space["Launch date"].str.split(" ").str[1:2]

launch_count_month = launch_month.value_counts()

print(launch_count_month.iloc[0:1])
```

```
Launch date
[November]    31
Name: count, dtype: int64
```

In [ ]:
```
# Ranking of months with the most to the fewest launches
print(launch_month.value_counts())

# Or using the sorting funtion
# sorted = launch_count_month.sort_values(ascending=False)
# sorted
```

```
Launch date
[November]     31
[August]       28
[September]    25
[October]      24
[July]         22
[January]      21
[May]          19
[December]     19
[February]     18
[March]        15
[April]        14
[June]         14
Name: count, dtype: int64
```

Supervillain challenge

1. Write a script that combines the tables showing supervillain debuts from the 30's through the 2010's
2. Write a script that ranks each decade in terms of how many supervillains debuted in that decade
3. Write a script that ranks the different comics companies in terms of how many supervillains they have, and display the results in a nice table (pandas dataframe)

In [97]:
```
#supervillandata = pd.read_html("https://en.wikipedia.org/wiki/List_of_comic_book_supervillain_debuts")

url_villan = "https://en.wikipedia.org/wiki/List_of_comic_book_supervillain_debuts"

# Add a User-Agent header so Wikipedia doesn't block it
headers = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"}
response = requests.get(url_villan, headers=headers)

# Pass the HTML text to pandas
supervillandata = pd.read_html(StringIO(response.text))

# combine all tables into data frame
df_supervillan = pd.concat(supervillandata, ignore_index = True)
df_supervillan.head()


# Fixing code -> Drop first three dataframes when joining
df_supervillan = pd.concat(supervillandata[3:], ignore_index=True) # Dropping first three dfs
df_supervillan.head()
```

Out[97]:

	Character / Team	Year Debuted	Company	Creator/s	First Appearance
0	Ultra-Humanite	1939 (June)	DC	Jerry Siegel, Joe Shuster	Action Comics (vol. 1) #13
1	Dr. Death	1939 (July)	DC	Bob Kane, Bill Finger	Detective Comics (vol. 1) #29
2	The Monk	1939 (September)	DC	Bob Kane, Bill Finger	Detective Comics (vol. 1) #31
3	The Claw	1939 (December)	Lev Gleason Publications	Jack Cole	Silver Streak Comics #1
4	NaN	NaN	NaN	NaN	NaN

In [ ]:
```
# 1. Write a script that combines the tables showing supervillain debuts from the 30's through the 2010's
```

In [ ]:
```
# 2. Write a script that ranks each decade in terms of how many supervillains debuted in that decade
```

In [26]:
```
# 3. Write a script that ranks the different comics companies in terms of how many supervillains they have, and dis
```